Project Report

Ticket Booking System

Testable Design, TDD, and Mocking Verification

Course & Topic	Software Testing - TDD, Test Doubles, and Mocking
System	Ticket Booking System
Main Class	BookingManager
Team Members	Aya Alkhateeb 2230887 - No.12 Alaa Alnajjar 2232195 - No.15 Saba Ramadeen 2038075 - No.1

Introduction

This report documents the design and testing approach used for the Ticket Booking System. The project focuses on designing a highly testable BookingManager class using constructor-based Dependency Injection and verifying its behavior using JUnit and Mockito.

The tests cover the required scenarios: successful booking, invalid input, and sold out event handling. Mockito mocks are used to isolate BookingManager from external dependencies such as payment processing, notification sending, and booking persistence.

Dependency Injection Explanation

The BookingManager class was designed using constructor-based Dependency Injection. Instead of creating concrete objects directly inside BookingManager, the class receives its dependencies through the constructor: IPaymentGateway, INotificationService, and IEventRepository. This design makes BookingManager loosely coupled because it depends on interfaces rather than concrete implementations.

This improves testability because the real external services can be replaced with mock objects during unit testing. For example, the payment gateway, notification service, and event repository can be mocked using Mockito, allowing the tests to control their behavior and verify how BookingManager interacts with them. Without Dependency Injection, BookingManager would be tightly connected to real implementations, making it harder to isolate the class and test its logic independently.

Testing and Mocking Approach

The testing phase was conducted to verify that the Ticket Booking System behaves correctly under the required scenarios. The tests were written using JUnit and Mockito. Mockito was used to create mock objects for the external dependencies: IPaymentGateway, INotificationService, and IEventRepository.

The Happy Path test verifies that when the input is valid, the event is not sold out, and the payment succeeds, both saveBooking() and sendConfirmation() are called exactly once using verify(..., times(1)). The Invalid Input test verifies that processPayment(), saveBooking(), and sendConfirmation() are never called when the booking input is invalid using verify(..., never()).

The Sold Out test verifies that isSoldOut() is called, but payment processing, booking persistence, and notification methods are not called when the event is sold out.

User Story	Scenario	Expected Behavior	Mockito Verification
US-01	Happy Path	Booking succeeds when input is valid, payment succeeds, and the event is not sold out.	saveBooking() and sendConfirmation() are called times(1).
US-02	Invalid Input	Booking fails before payment or persistence when input is invalid.	processPayment(), saveBooking(), and sendConfirmation() are called never().
US-03	Sold Out Event	Booking fails when the event is sold out.	isSoldOut() is called times(1); subsequent methods are called never().

Limitations of Mocking

Although mocking is useful for isolating the class under test, relying too heavily on mocks can create a risk that tests pass while the real system fails. This can happen when the mock behavior does not accurately represent the behavior of the real implementation. For example, a mocked payment gateway may always return a valid transaction ID, while a real payment service may fail because of network errors, invalid credentials, or service unavailability.

Another limitation is that mocks verify interactions, but they do not prove that the full system integration works correctly. In this project, the tests confirm that BookingManager calls processPayment(), saveBooking(), and sendConfirmation() correctly, but they do not confirm that a real database saves the booking or that a real notification service sends an email. Therefore, mocking is valuable for unit testing, but it should be supported by integration testing in larger systems.

Team Contribution Matrix

Team Member	Contribution	Related File(s)	Commit Evidence
Aya	Implemented the Happy Path test and updated BookingManager to make the happy path pass successfully.	BookingManager.java, BookingManagerTest.java	Add happy path test - red phase Implement happy path booking logic - green phase
Alaa	Implemented Invalid Input test cases and verified that payment, saving, and notification methods are never called for invalid input.	BookingManagerTest.java	Add invalid input booking test
Saba	Implemented the Sold Out test case and verified that payment, saving, and notification methods are never called when the event is sold out.	BookingManagerTest.java	Implemented sold-out logic and added comprehensive unit tests - Person 3
Team	Reviewed the final repository structure and prepared the ProjectReport.pdf file for submission.	ProjectReport.pdf	Add project report